SPEC 2111

Deep Learning and Neural Networks

Dr. Öznur Alkan

Week - 9

2026/2

Course Roadmap

PART-1: Introduction, Foundations & Essentials

- *Week 1 – Introduction & Course Overview*
- *Week 2 – Linear Regression*
  - *Regression concepts, gradient descent, loss functions*
- *Week 3 – Logistic Regression*
  - *Classification, sigmoid, cross-entropy loss*

PART-2: Neural Networks, Deep Learning and Training

- *Week 4 – Neural Network Essentials*
  - *Neurons, layers, forward propagation, activation functions*
- *Week 5 – Training, Refining Backpropagation and Deep Neural Networks (DNNs), Backpropagation, chain rule, gradient descent optimization*
- *Week 6 – Preventing Overfitting, Activation Functions, Optimizers*

PART-3: Specialized Architectures

- *Week 7 – Convolutional Neural Networks (CNNs)*
  - *Convolutions, pooling, image applications*
  - *Assessment-1*
- *Week 8 – Representations & Transfer Learning*
- *Week 9 – Recurrent Neural Networks & LSTM*
  - *RNNs, LSTM/GRU, sequence modeling*
- *Week 10 – Attention & Transformers*
  - *Attention mechanism, transformer architecture, NLP applications*

PART-4: Reinforcement Learning & Projects

- *Week 11 – Reinforcement Learning*
  - *Assessment-2*
- *Week 12 - Project Presentations & Discussions (Project Due)*

Course Roadmap (Remaining Weeks)

# Week	Date	Topic	Notes
Week 9	25th of March	RNNs & LSTMs	
Spring Break	1st of April	NO CLASS	
Spring Break	8th of April	NO CLASS	
No Class	15th of April	NO CLASS	
Week 10	22nd of April	Attention & Transformers	
Week 11	29th of April	Reinforcement Learning	**ASSESSMENT 2**
Week 12	6th of May	Discussions on New Topics, Project Presentations*	**PROJECTS DUE 4th of May@5pm**

*Reach out if you cannot share the data for the project work to get a slot for presentations

Week-9 / Agenda

Theme: RNNs and LSTMs

- Recap - Week 8
  - Text Representations and Transfer Learning
- RNNs
- LSTMs - A Special Type of RNNs
- LSTM Applications
- Lab Work

Text Representations

- Unlike images, text is not naturally structured for machine learning.
- Early Text Representations
  - BOW and One Hot Encoding
  - Suffers from huge sparse vectors, and having no way of encoding semantic similarity
- Distributed Word Vectors and Word2Vec
  - The meaning of each word is based on the distribution of terms with which it co-occurs
    - Word2Vec variants
      - Contextual Bag of Words (CBOW)
        - predicts the current word given context words within a specific window.
      - Skip Gram
        - predicts the surrounding context words within specific window given current word.

Building & Using Word Embeddings in Practice

Word embeddings can be build & used in **two main ways** in real NLP systems.

Option 1 — Pre-trained/Train Embeddings

- First we **train embeddings on a large text corpus** using a model such as **Word2Vec**.
- Example:
  - dog → [0.21, -0.34, 0.78]
  - cat → [0.18, -0.30, 0.75]
  - car → [-0.66, 0.44, -0.12]
- Then we **use these vectors as input features** for another task.
  - Example tasks:
    - sentiment analysis
    - document classification
    - question answering
    - recommendation systems

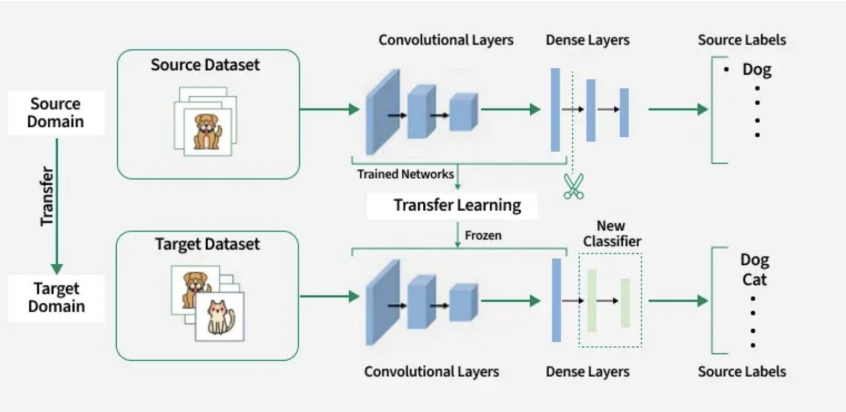
Option 2 — Learn Embeddings During the Task

- Instead of using pre-trained embeddings (like Word2Vec or GloVe), let the model learn embeddings directly from your dataset while training for the specific task.
- Example task: Sentiment analysis on movie reviews
  - Build a network for this task
- Embeddings are usually the weights of the first hidden layer (Embedding Layer)
- Can be extracted after training
- Frameworks like TensorFlow and Keras make this very easy using an Embedding Layer.
  - Embedding is a layer type, just like: Dense layer, Convolution layer, Pooling layer

Transfer Learning

RECAP

- Transfer Learning = using a model trained in one domain for a different domain
  - Typical workflow:
    - Train on large dataset
    - Reuse general-purpose layers
    - Replace and train new task-specific head
      - Head = final layers of a neural network that are specific to the original task.
      - Usually includes: Fully connected layers and output layer



Key Idea:

- Neural networks, like software modules, can be reused in appropriate scenarios.
- Early layers = general-purpose (edges, textures, embeddings)
- Head = task-specific (predicts original 1000 classes)

Freeze vs Fine Tuning

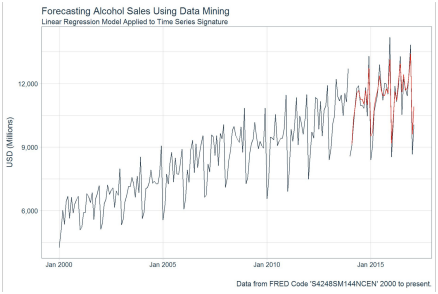
When we import elements of the network, we can freeze their weights (not let them change) or allow them to be changed as part of the process of training on new data - this is referred to as fine tuning.

Sequential Data & RNNs

Time Series & Sequential Data

- What is Sequential Data?
  - Sequential data is any data where order matters.
  - Each element depends on previous elements
  - sequence must be processed in order
  - Examples:
    - Text → “The cat sat…”
    - Speech / audio signals
    - Video frames
    - DNA sequences
- Key idea: Changing the order changes the meaning

Sequential Categorial Data

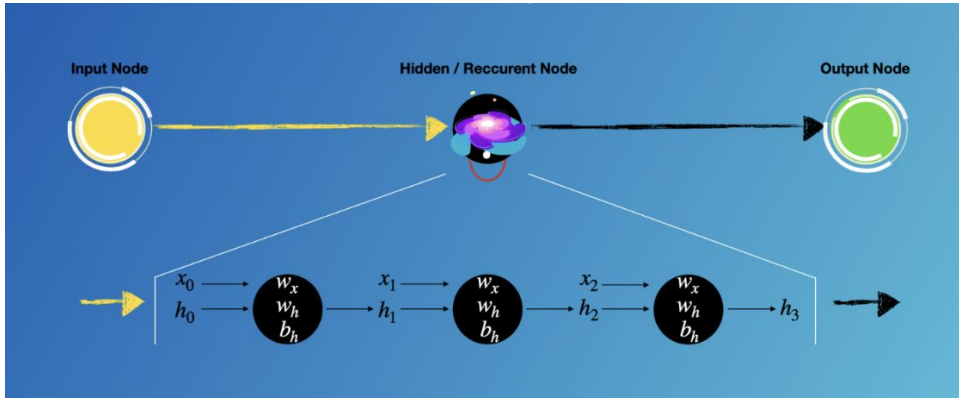


Real valued time-series data

- What is Time-Series Data?
  - Time-series data is a special type of sequential data where:
    - Data points are indexed by time
    - Examples:
      - Stock prices
      - Temperature readings
      - Sensor data (IoT devices)
  - Key idea: Observations are taken generally at regular (time intervals

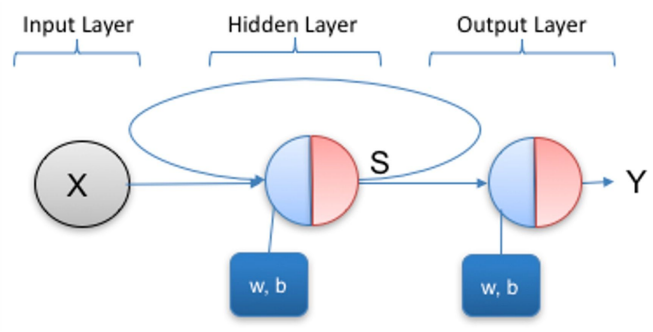
RNNs

- RNNs are neural networks designed for sequential data
- offers flexible input–output structures.
- Traditional (Vanilla) Neural Networks:
  - Take fixed-size input
  - Produce fixed-size output
  - Cannot naturally handle sequences
- Key Advantage of RNNs
  - Can process:
    - Sequences of inputs
  - Output at each step depends on:
    - Current input
    - All previous inputs (memory)



A Basic Recurrent Neural Network

- A Recurrent Neural Network (RNN) is a neural network where:
  - hidden state output at time step *i* is fed back into the network along with the next input i+1
  - this creates a memory mechanism
- Key Idea
  - Output at time t depends on:
    - Current input
    - All previous inputs
- Think of an RNN like a running summary of everything it has seen so far.
  - At each time step it
    - reads a new word/input
    - updates its memory (hidden state)
  - That hidden state is not just the last input
    - It's a compressed summary of all previous inputs



Let's say our sequence is: "I love deep learning"

RNN process one word at a time:

t = 1 ("I")

- Hidden state = summary of: "I"

t = 2 ("love")

- RNN takes:
  - previous hidden state (summary of "I")
  - current input ("love")
- New hidden state = summary of: "I love"

t = 3 ("deep")

- Takes: summary of "I love"
- new word "deep"
- New hidden state
  - summary of: "I love deep"

t = 4 ("learning")

- Hidden state
  - summary of: "I love deep learning"

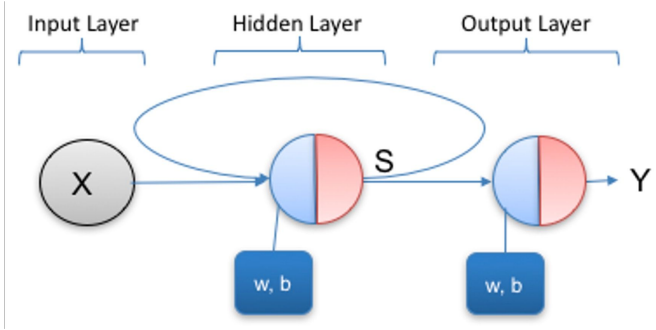
At time **t**, the hidden state contains:

> information from $x_1, x_2, \ldots, x_t$

So the output at time **t** depends on **all previous inputs** because:

- They've been **accumulated into the hidden state**

Vanilla RNN



- single neuron in an input layer
- a single neuron in the output layer
- and a state variable that is one neuron wide.

- assume that
  - the input to this network, X, is a stream of values $\in R$
  - the output from the network, Y, is a stream of values $\in \{0,1\}$
- State is the output of the hidden RNN
  - passed to the output layer
  - passed to the hidden layer itself.
- This visualization simplifies temporal behavior of RNN

- For clarity, it is useful to think of the network as being unrolled over time to process sequences of inputs.

Unrolling Input

- Example: “I do not like this movie”
- **One input sentence = one sequence (one row of time steps)**

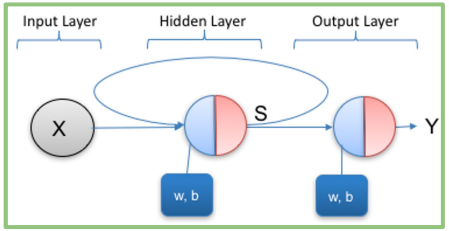
We represent it as:

```
Time step:   t1   t2   t3   t4   t5   t6
Input:       I    do   not  like this movie
```

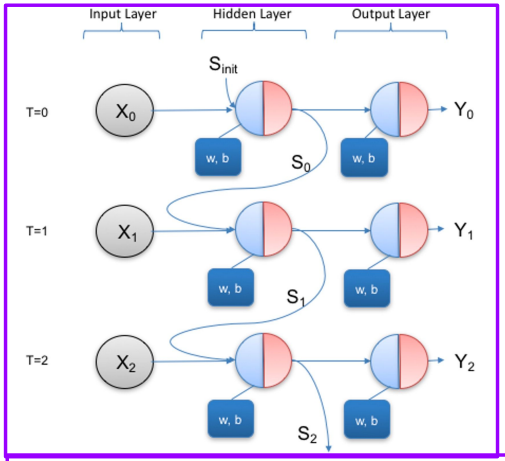
Each word = one **time step**

RNN processes it like:

- x_1 = “I”
- x_2 = “do”
- x_3 = “not”
- …
- x_6 = “movie”



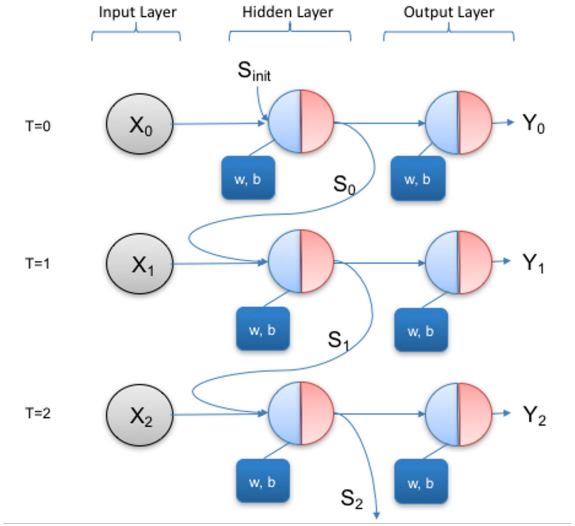
UNROLL



Instead of thinking of one loop (“one neuron processing everything repeatedly)

we “unroll” it such that the **same neuron (same weights)** copied across time steps

Unrolling RNN Through Time



- Each time step is shown as a separate copy of the same network
- All copies share the same parameters (weights)
- Step by step operation:

at t = 0

- Input: X_0
- Hidden state: initialized (usually 0)
- Output: Y_0
- Hidden state S_0:
  - Sent to output
  - Passed to next timestep $t = 1$

at t = 1

- Input: X_1
- Hidden state uses:
  - X_1
  - Previous state S_0

👉 Output depends on:

- Current input
- Previous input (via S_0)

at t = 2

- Input: X_2
- Hidden state uses:
  - X_2
  - S_1

👉 But S_1 already contains information from:

- X_1
- X_0

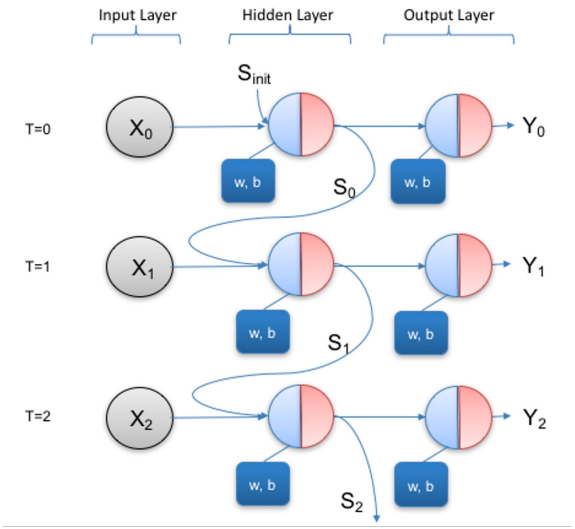
- At time $t = 2$, output depends on:
  - X_2 (current input)
  - X_1, X_0 (past inputs via hidden state)

S_T is concatenated with the actual input X_{T+1}.

At time T=1 we don't have a true hidden state from a previous step to concatenate with our input X_0

- In this case we typically use an initial value for S which we set to 0.

Hidden State



$$S_t = tanh((W * (X \oplus S_{t-1})) + b)$$

tanh used as the activation function

Hidden State

- Output of the hidden RNN unit
- Denoted as S
- It acts as:
  - a memory of past inputs
  - an additional input coming from the previous time step
- Passed to the next time step

Hidden State Computation is similar to a standard feedforward layer

- input at time t consists of: Current input X_t , Previous hidden state S_{t-1}
- These are concatenated together
- The combined vector is then:
  - Multiplied by weights
  - Added to bias
  - Passed through the tanh activation function

Activation Function in RNNs

- tanh is usually preferred over ReLU in RNNs
- ReLU can cause
  - exploding hidden states
    - No upper bound → values can keep growing each timestep
    - Over many steps hidden state may blow up
  - instability over time
    - Small changes can accumulate
  - output 0 for negative inputs
    - Once a neuron stays negative → it may stop contributing
- ReLU works well in CNNs because there's no repeated recurrence over time
  - the "explosion across timesteps" problem doesn't exist the same way.
  - In RNN however the same weights applied repeatedly over many timestamps

`tanh`

- Output range: **[-1, 1]**
- Hidden state stays **bounded**

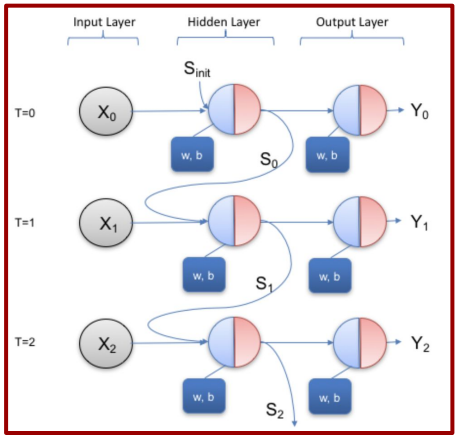
ReLU

- Output range: **[0, ∞)**
- Hidden state can grow **unbounded**

PRACTICAL TIP:

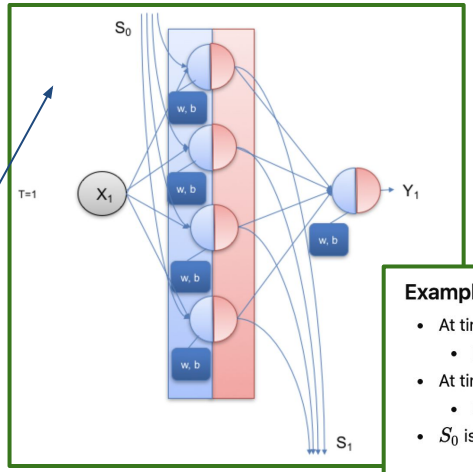
- For **vanilla RNNs**, **tanh or sigmoid** is almost always used
- ReLU works better in **feedforward or CNNs** where there is no recurrent multiplication
- If you really want ReLU in RNNs, you need:
  - careful weight initialization
  - gradient clipping (scaling gradients to avoid explosion)

Hidden StateSize



Increasing hidden state size:

- Increases **memory capacity**
- Allows learning of **richer temporal features**



Example: Hidden State Size = 4

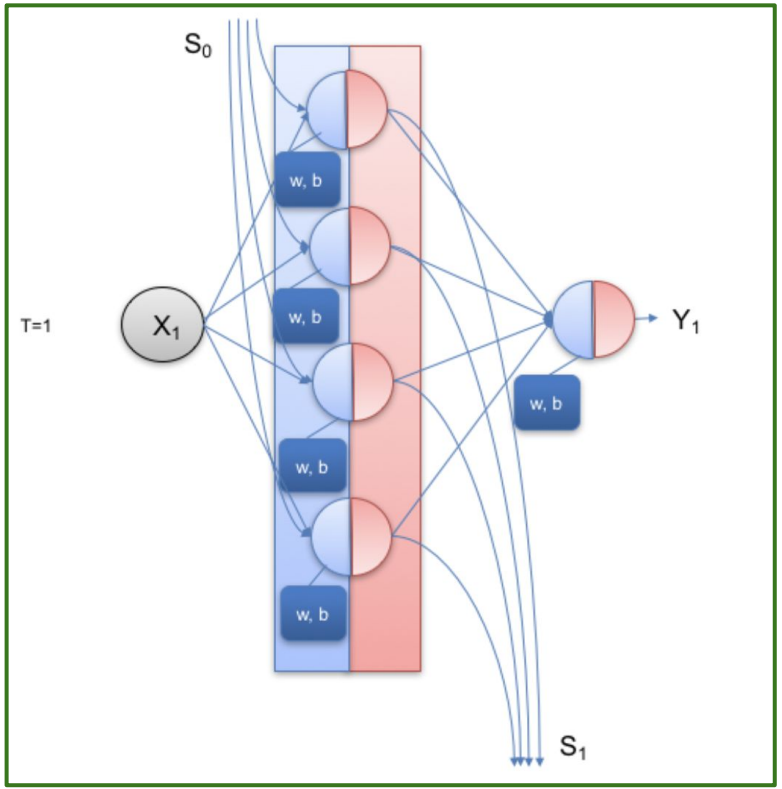
- At time $t = 0$:
  - Hidden state: $S_0 \in \mathbb{R}^4$
- At time $t = 1$:
  - New state: $S_1 \in \mathbb{R}^4$
- S_0 is passed forward to compute S_1

Basic Case: we assumed hidden state cell has 1 hidden neuron

- This means only one value is carried across time
- A single value can only store very limited information
- Use a larger hidden state (vector instead of scalar)
- Hidden state size = number of hidden neurons

If your cell has **4 neurons (units)** → the state is a **vector of size 4**

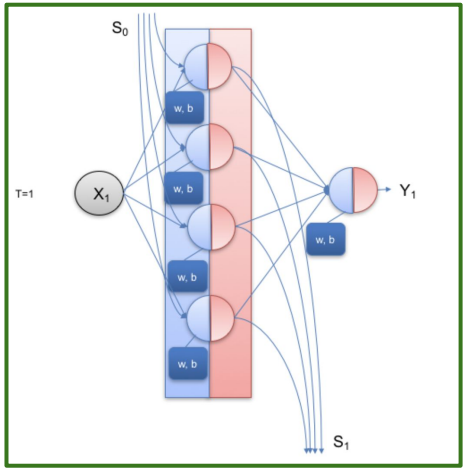
RNN Cell



RNN Cell Concept

- An RNN cell:
  - **Contains multiple neurons internally**
  - Performs:
    - Linear transformation (logits)
    - Non-linear activation
- Increasing hidden state size/RNN cell size:
  - Increases **memory capacity**
  - Allows learning of **richer temporal features**

Hidden State vs RNN Cell



- Hidden state acts as the network's memory.
- RNN cell computes the hidden state
- Cell → Process
- Hidden State →Result

Hidden State S_t

- The output of the RNN at time t
- A vector of values
- Represents the memory of the network

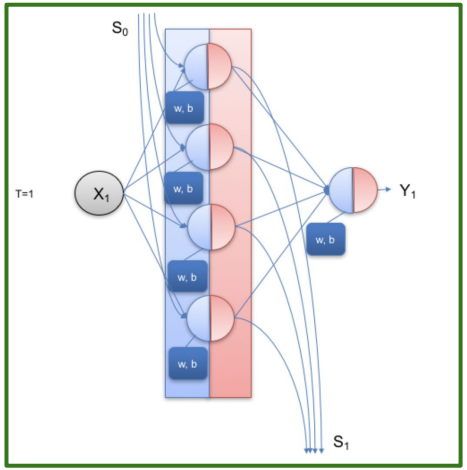
Conceptually: "What the network remembers at time t"

RNN Cell

- The computation unit (like a function/module)
- Takes:
  - X_t (input)
  - S_{t-1} (previous state)
- Produces:
  - S_t (new hidden state)

Conceptually: "The machine that updates the memory"

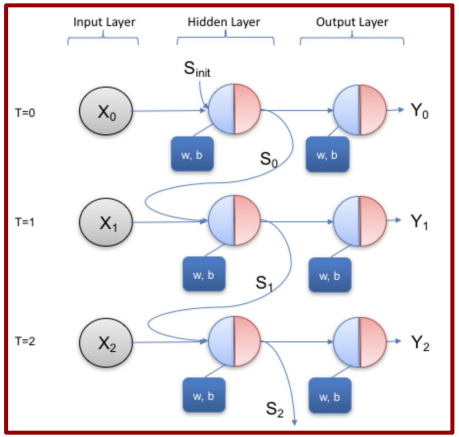
Hidden State vs RNN Cell



- By convention, we do not illustrate each individual neuron within an RNN cell
- Instead, a square is used to represent the entire RNN cell
- The cell is often split into two parts to indicate that:
  - A logit (linear transformation) is computed first
  - A non-linearity (activation function) is then applied

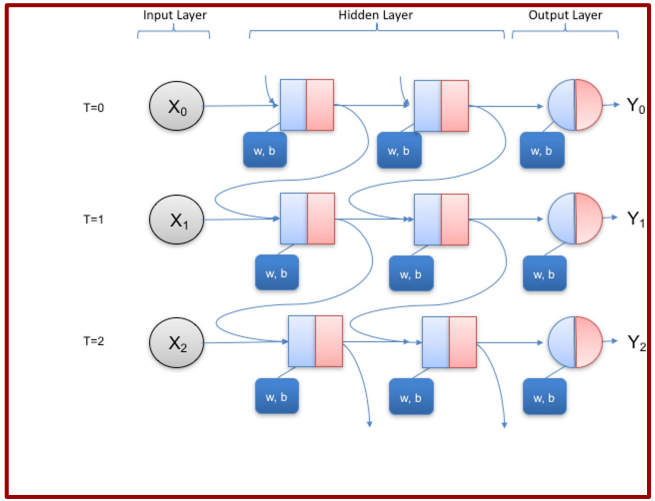
Multiple RNN Layers

Single-layer RNN



- RNN layer
  - only recurrent cells
- You can stack or mix RNN layers with normal layers in a network

Multi-layer RNN



- In practice, RNNs can be stacked into multiple layers
- output of one layer serves as the input to the next.

RNN Input

- RNN input is **not a single vector**
  - It is a **sequence of vectors over time**
- Each timestep is **not just one number (usually)**
  - It is a **set of features**
- ex: Temperature readings
  - x_t=[temperature,humidity,pressure]
    - Each timestep has **multiple features**
  - Final input shape: (batch_size, timestamp, features)

Suppose we have **3 features** measured every hour:

Time	Temperature (°C)	Humidity (%)	Pressure (hPa)
t1	20	55	1012
t2	21	57	1011
t3	21	60	1010
t4	22	62	1009

Each time step is a **vector of features**:

$$x_1 = [20, 55, 1012], \quad x_2 = [21, 57, 1011], \ldots$$

- Suppose we have **2 days of data**, each day has 4 hours:
- Batch of sequences shape: [batch_size, time_steps, features] = [2, 4, 3]

Example tensor:

```
[
  [[20,55,1012], [21,57,1011], [21,60,1010], [22,62,1009]],  # Day 1
  [[18,50,1013], [19,53,1012], [20,55,1011], [21,58,1010]]   # Day 2
]
```

- Each row = **one sequence**
- Each column along time = **one time step**
- Each innermost vector = **features at that time step**

RNN processing

Example tensor:

```
[
  [[20,55,1012], [21,57,1011], [21,60,1010], [22,62,1009]],  # Day 1
  [[18,50,1013], [19,53,1012], [20,55,1011], [21,58,1010]]   # Day 2
]
```

- Each row = **one sequence**
- Each column along time = **one time step**
- Each innermost vector = **features at that time step**

1. **t1 → RNN cell** sees `[20,55,1012]` → produces hidden state `h1`
2. **t2 → RNN cell** sees `[21,57,1011] + h1` → produces `h2`
3. **t3 → RNN cell** sees `[21,60,1010] + h2` → produces `h3`
4. **t4 → RNN cell** sees `[22,62,1009] + h3` → produces `h4`

Handling Variable-Length Inputs (RNN vs CNN)

RNNs

- **Can process variable-length sequences (in theory)**
  - Works step-by-step over time
  - No inherent restriction on sequence length
- **In practice (training):**
  - Inputs must be converted into **fixed-size tensors**
  - Therefore, sequences are made equal length using:
    - **Padding** (add dummy values)
- **Masking**
  - Used to tell the RNN: “Ignore padded timesteps”
  - Ensures padding does not affect learning

```
X = np.array([
    [[1.0], [2.0], [0.0], [-999.0], [-999.0]],  # padded sequence
    [[3.0], [0.0], [4.0], [5.0], [6.0]]         # full sequence
])

# Dummy target (for demonstration)
Y = np.ones((2, 5, 1))

# ------------------------------------------------------------
# Model with masking
# 👉 Masking tells RNN to IGNORE padded values (-999)
# ------------------------------------------------------------

model = Sequential()
model.add(Masking(mask_value=-999.0, input_shape=(5, 1)))
model.add(SimpleRNN(8, return_sequences=True))
model.add(Dense(1, activation='sigmoid'))
```

CNNs

- **Require fixed-size inputs**
  - Especially for images (e.g., 224×224×3)
  - Because convolutions and fully connected layers expect consistent dimensions
- Common solution:
  - **Resize** or **crop** inputs

RNN Outputs: Different Usage Modes

1. Final Output Only (Many-to-One)

- Use only the last output of the RNN
- Output corresponds to:
  - Final time step t= T

✔ When to use:

- Sequence classification tasks
  - e.g., sentiment analysis

Model summarizes the entire sequence into one prediction

2. Output at Every Time Step (Many-to-Many)

- RNN produces an output at each time step
- Outputs: $Y_1, Y_2, \ldots, Y_T$

✔ When to use:

- Sequence labeling tasks
  - e.g., part-of-speech tagging
    - "dog runs" → [Noun, Verb]

Each output depends on:

- Current input
- Previous context (hidden state)

Why Use All Outputs?

- Provides richer information than final output alone
- Downstream models can:
  - Use intermediate outputs
  - Make more informed decisions

Coding - when return_sequence = False

- Deep learning frameworks (e.g., TensorFlow, PyTorch) allow:
  - Returning only final output
  - Returning full sequence of outputs
- This behavior is configurable

We have:

- 2 sequences (batch)
- Each sequence has 3 timesteps
- Each timestep has 2 features

```
# Toy input: batch_size=2, timesteps=3, features=2
X = np.array([[[1, 2], [3, 4], [5, 6]],
              [[7, 8], [9,10], [11,12]]], dtype=float)
```

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import SimpleRNN, Dense

model = Sequential()

# RNN layer (returns last hidden state only)
model.add(SimpleRNN(
    units=4,
    activation='tanh',
    return_sequences=False,
    input_shape=(10, 5)
))

# Output layer
model.add(Dense(1, activation='sigmoid'))

model.summary()
```

For each sequence:

Sequence 1:

```
t=1 → S1 (4 values)
t=2 → S2 (4 values)
t=3 → S3 (4 values)
```

Sequence 2:

```
t=1 → S1 (4 values)
t=2 → S2 (4 values)
t=3 → S3 (4 values)
```

3 hidden states per sequence
Each hidden state has **4 neurons**

Output from RNN layer to dense layer

```
[
  [0.2, −0.1, 0.5, 0.7],   ← sequence 1 (S3)
  [0.9,  0.3, −0.2, 0.4]   ← sequence 2 (S3)
]
```

Output layer (One prediction per sequence)

```
Input to Dense: (2, 4)
Output:         (2, 1)
```

Coding - when return_sequence = True

```
# Toy input: batch_size=2, timesteps=3, features=2
X = np.array([[[1, 2], [3, 4], [5, 6]],
              [[7, 8], [9,10], [11,12]]], dtype=float)
```

We have:

- 2 sequences (batch)
- Each sequence has 3 timesteps
- Each timestep has 2 features

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import SimpleRNN, Dense

model = Sequential()

# RNN layer (returns output at every timestep)
model.add(SimpleRNN(
    units=4,
    activation='tanh',
    return_sequences=True,
    input_shape=(10, 5)
))

# Output layer applied at each timestep
model.add(Dense(1, activation='sigmoid'))

model.summary()
```

For each sequence:

Sequence 1:

```
t=1 → S1 (4 values)
t=2 → S2 (4 values)
t=3 → S3 (4 values)
```

Sequence 2:

```
t=1 → S1 (4 values)
t=2 → S2 (4 values)
t=3 → S3 (4 values)
```

3 hidden states per sequence
Each hidden state has **4 neurons**

RNN always computes all hidden states, but return_sequences decides how many of them are returned.

Output from RNN layer to dense layer (All hidden states returned)

```
[
  [  # Sequence 1
    [0.1, 0.2, 0.3, 0.4],   ← S1
    [0.2, 0.1, 0.5, 0.6],   ← S2
    [0.2,-0.1, 0.5, 0.7]    ← S3
  ],
  [  # Sequence 2
    [0.5, 0.6, 0.7, 0.8],   ← S1
    [0.7, 0.2, 0.1, 0.3],   ← S2
    [0.9, 0.3,-0.2, 0.4]    ← S3
  ]
]
```

```
(2, 3, 4)
```

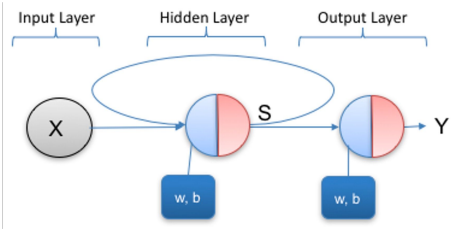
- 2 sequences
- 3 timesteps
- 4 neurons per hidden state

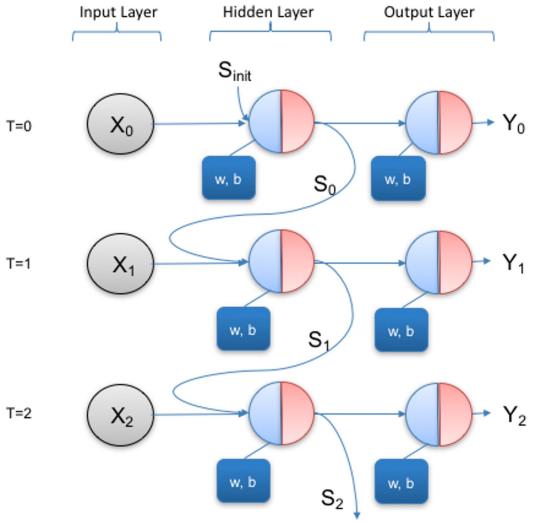
Output layer (**One prediction per timestamp**)

```
Input to Dense: (2, 3, 4)
Output:         (2, 3, 1)
```

Aside: Unrolling the Vanilla RNN for Training

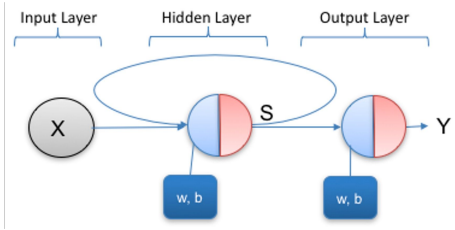
Training an RNN involves unrolling it through time and propagating errors backward across multiple timesteps

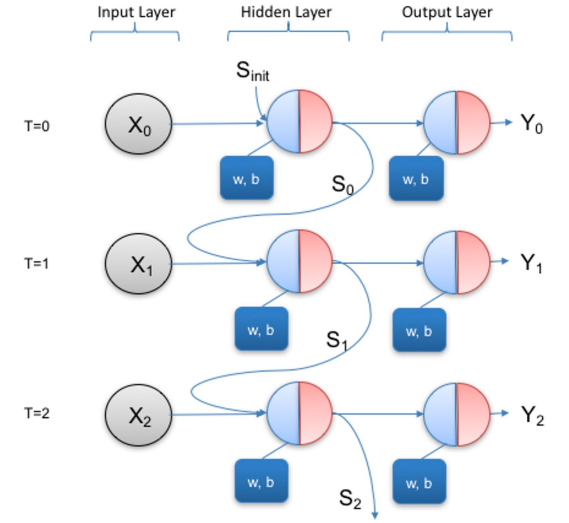




- Why Unrolling is Important
  - Makes its operation across time explicit
  - Is not just for visualization
  - Is essential for both inference and training
- Forward Pass (Inference)
  - Given fixed model parameters:
  - Input sequence is processed one time step at a time
  - At each timestep t:
    - Hidden state is updated
    - Output is produced
  - Hidden state St is passed to time t+1
    - Concatenated with the next input
  - Produces valid outputs for the entire sequence
- Training the RNN
  - Loss is computed over the entire sequence
  - Not just the final output:
    - Error is calculated at each timestep
  - Total loss:
    - Sum (or average) of errors across all timesteps

Backpropagation





- RNN must be **unrolled through time**
- Loss is computed across all timesteps
- Gradients flow backward through time
- Truncated BPTT is used in practice

Backpropagation Through Time (BPTT)

- Errors are propagated:
- From later timesteps → earlier timesteps
- Enables learning of:
  - Temporal dependencies

How Far Back Should We Propagate Errors?

- Case 1: No Backpropagation
  - No learning across time
  - Cannot capture sequence dependencies
- Case 2: Full Backpropagation
  - Errors propagated through all timesteps
  - Allows learning long-term dependencies
  - Problems:
    - High computational cost (time & memory)
    - Vanishing / exploding gradients
- Practical Solution: Truncated BPTT
  - Limit backpropagation to a fixed number of timesteps
  - Trade-off:
    - More efficient
    - Reduces gradient issues
    - May miss very long-term dependencies

How Keras Handle Backpropagation Through Time

A. Fixed-length sequences

- Chop your data into **short sequences** of length `truncated_steps`.

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import SimpleRNN, Dense

model = Sequential()
model.add(SimpleRNN(16, input_shape=(20, 3)))  # sequence length = 20
model.add(Dense(1))
model.compile(optimizer='adam', loss='mse')
```

- 16 → number of **hidden units** (size of hidden state)
- `input_shape` → **shape of a single sequence** (not including batch size)
- Here, RNN back propagates through **20 time steps only**.
- Longer sequences are broken into chunks
  - to chunks of 20

B. Stateful RNNs

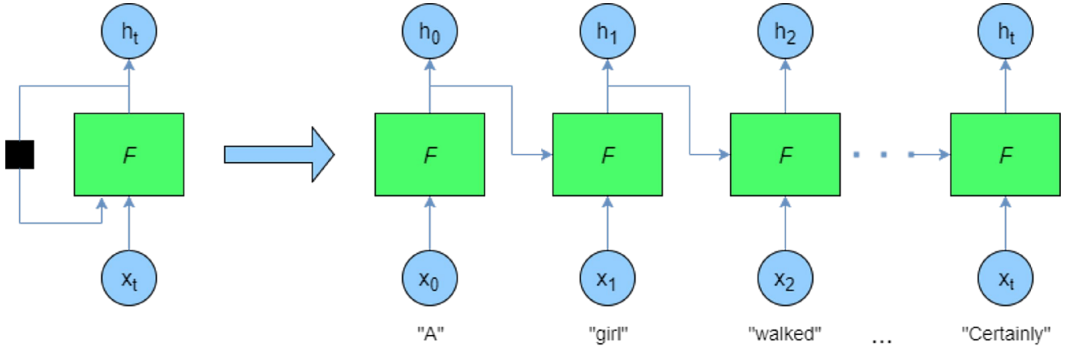
- When `stateful=True`, hidden states are **kept across batches**.
- You feed the network **in small window** and reset states periodically.

```
model = Sequential()
model.add(SimpleRNN(16, batch_input_shape=(batch_size, 20, 3), stateful=True))
model.add(Dense(1))
model.compile(optimizer='adam', loss='mse')

# Training loop
for epoch in range(epochs):
    for batch in dataset:
        model.train_on_batch(batch_X, batch_y)
    model.reset_states()  # truncate BPTT between epochs
```

The **hidden states are carried forward** for continuity `reset_states()` effectively **cuts the gradient flow**, achieving TBPTT.

A Horizontal View on a Similar Model



- RNN processes sequences **one word at a time**
- Hidden state = **memory of the words seen so far**
- Output can be produced at each timestep or only at the last word

At each timestep, the RNN sees:

- Current word+Previous hidden state

Timestep	Input X_t	Previous Hidden State S_{t-1}	New Hidden State S_t	Notes
t = 0	"A"	S_0 = 0 (initial)	S_1 = f("A", S_0)	First word, hidden state starts memory
t = 1	"girl"	S_1	S_2 = f("girl", S_1)	Hidden state now remembers "A girl"
t = 2	"walked"	S_2	S_3 = f("walked", S_2)	Hidden state now contains "A girl walked"

Simple Scenario: Alarm Tripping (From notebook)

Scenario Description

We want to create a Trip Alarm for Negative Inputs:

- The alarm outputs 0 as long as input values are positive.
- If a negative value appears, the output switches to 1 and remains 1 for the rest of the sequence.

This demonstrates the memory property of RNNs: the network remembers that a negative value has occurred

Generate Sample Dataset

- Generate sequences
- Input values range from **-1 to 20**.
- Target output (Y) is generated as:
  - All zeros until a negative input occurs.
  - Switch to 1 at the time step of the negative input and keep it 1 till the end.

Example dataset

```
[[13 15 17 14 18 10 16  3 12 13 -1 10 10  3  5 11 16 11 19 15]]
[[ 0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  1.  1.  1.  1.  1.  1.  1.  1.
   1.  1.]]
[ 3  9  5 -1  6 14 10  7 12  1  2  5  4  8 18 18  8 15 -1  4]
[ 0.  0.  0.  1.  1.  1.  1.  1.  1.  1.  1.  1.  1.  1.  1.  1.  1.  1.
  1.  1.]
```

Simple Scenario: Alarm Tripping (From notebook)

```
from tensorflow.keras import layers, Sequential

batch_size = 100
state_size = 8
num_classes = 1

model = Sequential()
model.add(layers.SimpleRNN(state_size, return_sequences=True, input_shape=(20,1)))
model.add(layers.Dense(num_classes, activation='sigmoid'))

model.compile(loss="binary_crossentropy", optimizer="adam", metrics=["accuracy"])
model.summary()
```

- RNN Layer: SimpleRNN with a hidden state size of 8
- Output Layer: Dense with 1 neuron, sigmoid activation for binary classification
  - The output is a single number between 0 and 1
  - This represents: P(class = 1)
- return_sequences=True ensures outputs at all timesteps are produced.
- Loss is computed for every timestep, not just the final one.

Example Output

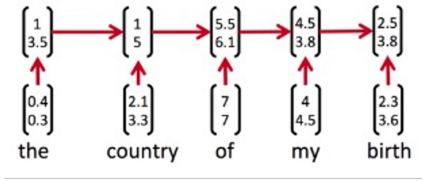
```
[[12.  3. 16. -1.  8. 17.  9.  3. 17. 10. 16. -1.  3. 17.  5. 19. 17. 12.
   2. 10.]]
[[3.3089519e-04 6.1523914e-04 9.6350908e-05 9.9246788e-01 9.9904907e-01
  9.9925864e-01 9.9953276e-01 9.9955821e-01 9.9935770e-01 9.9953091e-01
  9.9942940e-01 1.0000000e+00 9.9959421e-01 9.9936098e-01 9.9955291e-01
  9.9925590e-01 9.9937165e-01 9.9950957e-01 9.9955595e-01 9.9952567e-01]]
```

Observations:

- When all inputs are positive → output < 0.5
- When a negative input occurs → output rises > 0.5 and remains high

RNN **remembers** the negative input via its **hidden state**, demonstrating **temporal memory**.

Recurrent vs Recursive Neural Networks

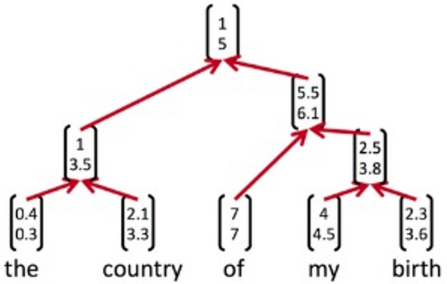


Recurrent Neural Networks (RNNs)

- A Recurrent Neural Network (RNN) reuses its hidden state over time
  - At each timestep:
    - The model takes:
      - Current input x_t
      - Previous hidden state S_{t-1}
    - Produces:
      - New hidden state S_t
      - Output y_t

Key Idea

- Information flows through time (sequentially)
- Same network is applied at each timestep
- No hierarchical structure



Recursive Neural Networks (RecNN)

- A Recursive Neural Network applies the model over a structure (not time)
- Output of the model is fed back as input to build higher-level representations

Key Idea

- Information flows through structure (hierarchy/tree)
- Model is applied repeatedly to combine smaller parts into larger ones

Intuition: Build meaning from parts

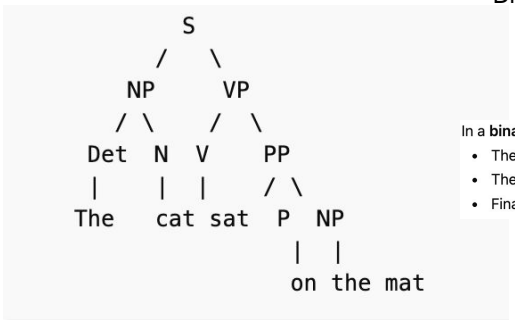
RecNN

ex: mathematical / logical expressions

- Task: Evaluate a tree-like expression (3 + (2 * 5)).
- Why RecNN:
  - Leaves = numbers (3, 2, 5)
  - Internal nodes = operations (+, *)
  - Recursive combination → root node gives final answer.

ex: parsing and syntax analysis

- Task: Evaluate whether a sentence is grammatically correct
- Input
  - entire sentence (or phrase) is represented as a tree, typically a parse tree.
  - Each leaf node (word) is converted into a vector (embedding).
  - Internal nodes don't have raw inputs—they are computed recursively from their children.



- Keras (and TensorFlow) does not have a built-in Recursive Neural Network layer like it does for RNNs.
- RecNNs are more specialized because they operate on tree structures, which aren't just sequences.
- You typically have to implement them manually using tf.keras.layers.Layer or PyTorch-style custom layers.

Binary Tree RecNN Node Combination

$$h_{parent} = f(W \cdot [h_{left}; h_{right}] + b)$$

In a **binary tree**, if you have n leaf nodes:

- The first internal layer has $n/2$ nodes (each combines 2 leaf embeddings)
- The next internal layer has $n/4$ nodes, and so on
- Finally, the **root node** is a single node representing the whole sentence

Why we need RecNN

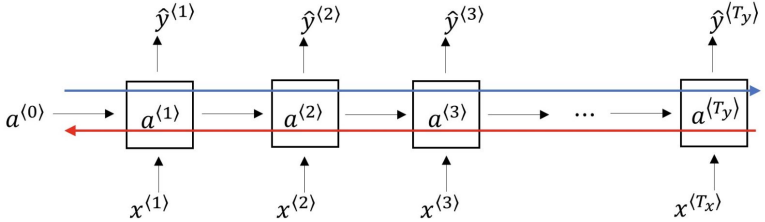
RecNNs mirror the tree structure of the data.

- Each parent node combines its children using the same learned function.
- This mirrors natural hierarchies like:
  - Parse trees in sentences
  - Mathematical expressions
  - Hierarchical object compositions in images

RecNN structures **the computation to follow the tree**, which no vanilla RNN or feedforward network does automatically. Without that structure, a network could struggle to learn the hierarchical relationships efficiently.

- Even if an RNN could, in theory, learn some “tree-like” behavior by turning neurons on/off, it would be **inefficient and unreliable** because it isn’t structured to reflect the hierarchy explicitly.
- Example: “The dog that chased the cat barked loudly.”
  - A **sequence RNN** sees words one after another
  - it may struggle to associate “dog” with “barked”
    - because “that chased the cat” is in between.
  - A **RecNN** builds a parse tree, combining sub-phrases first:
    - Combine “the dog” → hidden state 1
    - Combine “that chased the cat” → hidden state 2
    - Combine the two → parent phrase → final output
  - This respects **syntactic hierarchy**, making it easier to learn the correct relationship.

Why Vanilla RNNs Sometimes Struggle



The dog, which ran out …, has come back

The dogs, which ran out …, have come back.

Vanishing Gradient

- the word "*Dog*" at the very beginning of the sentence influences the word "*has*" which is at the very end.
  - the sentence in between can get longer
  - Ex: "*The dog, which ran out of the door and sat on the neighbor's porch for three hours and roamed the street for a couple of hours more, has come back.*"
  - So, for a word that's almost at the end of a very long sentence to be influenced by a word which is almost at the beginning of that sentence, is what we call a "long-term dependency".
- basic RNNs are not very good at handling such long-term dependencies, mainly due to the Vanishing Gradient Problem.

The Challenge: "Travelling Back in Time"

- Error signals for early timesteps get **diluted** as they move backward through the network.
- This happens because each time step adds more layers during backpropagation.
- As a result, the network **struggles to learn dependencies that span many timesteps.**

- Vanilla RNNs: prone to **vanishing gradients**
  - struggle with long-term dependencies
- Exploding gradients: rarer, but possible
- LSTMs/ largely mitigate both problems with **gates**

Solution: Long Short Term Memory

Long Short-Term Memory (LSTM) Networks

What are LSTMs?

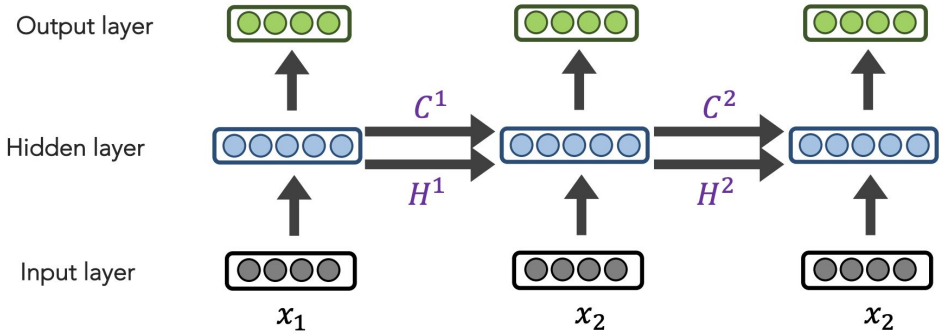
- Special type of RNN designed to learn long-term dependencies.
- Unlike vanilla RNNs, LSTMs remember information for long periods easily.
  - Vanilla RNN: hidden state overwritten every step
    - Hidden state is completely updated at each step.
    - There's no mechanism to preserve long-term memory, so older information can be overwritten or forgotten quickly.
    - This is why vanilla RNNs struggle with long-term dependencies.
  - LSTM: cell state allows part of memory to be carried forward intact
    - "not forgetting"

LSTM cell has both a hidden state and a cell state, whereas a vanilla RNN only has a hidden state.

The cell state is designed to:

- carry long-term information
- avoid vanishing gradients

Long Short-Term Memory (LSTM) Networks



- LSTM network (or LSTM layer)
  - a sequence-processing layer made of many LSTM cells, applied across time steps.
- LSTM cell
  - the recurrent unit that handles one time step:
    - Contains gates (input, forget, output)
    - Updates its cell state and hidden state

At each each time step t, we have a hidden state h^t and cell state c^t:

- Both are vectors of length n
- cell state c^t stores long-term info

You can **stack LSTM layers** to make a deep network:

```
Input sequence → LSTM layer 1 → LSTM layer 2 → Dense → Output
```

- Each layer is **made of LSTM cells**

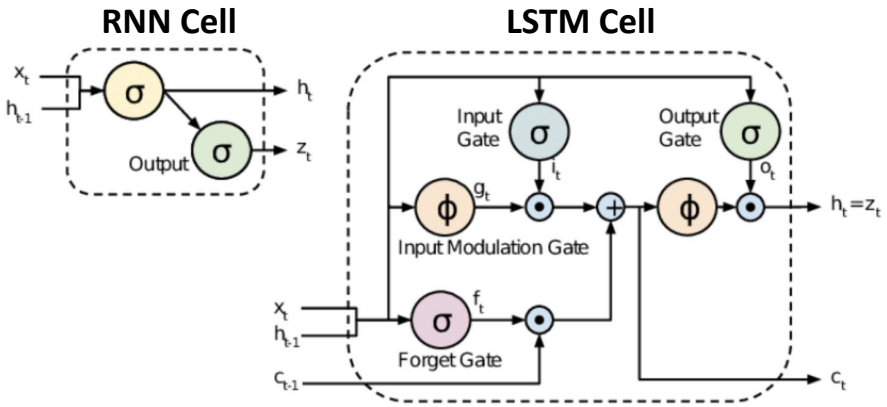
LSTM Cell

LSTM cell = the recurrent unit that handles one time step:

- *Contains gates (input, forget, output)*
- *Updates its cell state and hidden state*

At each timestep, an LSTM cell:

1. **Receives inputs**
   - current input x_t
   - previous hidden state h_{t-1}
   - previous cell state C_{t-1}
2. **Decides what to remember and forget** using gates
3. **Updates its memory (cell state)**
4. **Produces an output (hidden state)**



Hidden State

- output of the LSTM at time step t
- summarizes the information **from the current step and previous steps**
- **exposed to the next layer**

Cell State

- internal memory of LSTM.
- Carries **long-term information** across many time steps
- Updated via **forget and input gates**
- Part of it can be preserved **without modification** if gates allow

Internal components of an LSTM cell

1. Forget gate

Decides what to keep from the past:

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f)$$

What should I erase?

2. Input gate

Decides what new information to store:

$$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i)$$

What new info should I store?

3. Candidate memory

Proposes new content to add:

$$\tilde{C}_t = \tanh(W_c[h_{t-1}, x_t] + b_c)$$

My long term memory

4. Output gate

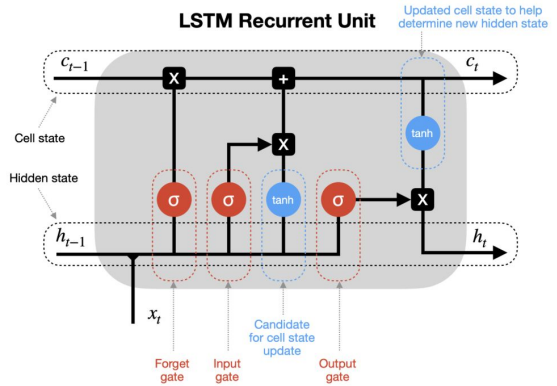
Controls what part of memory is exposed:

$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o)$$

What should I reveal?

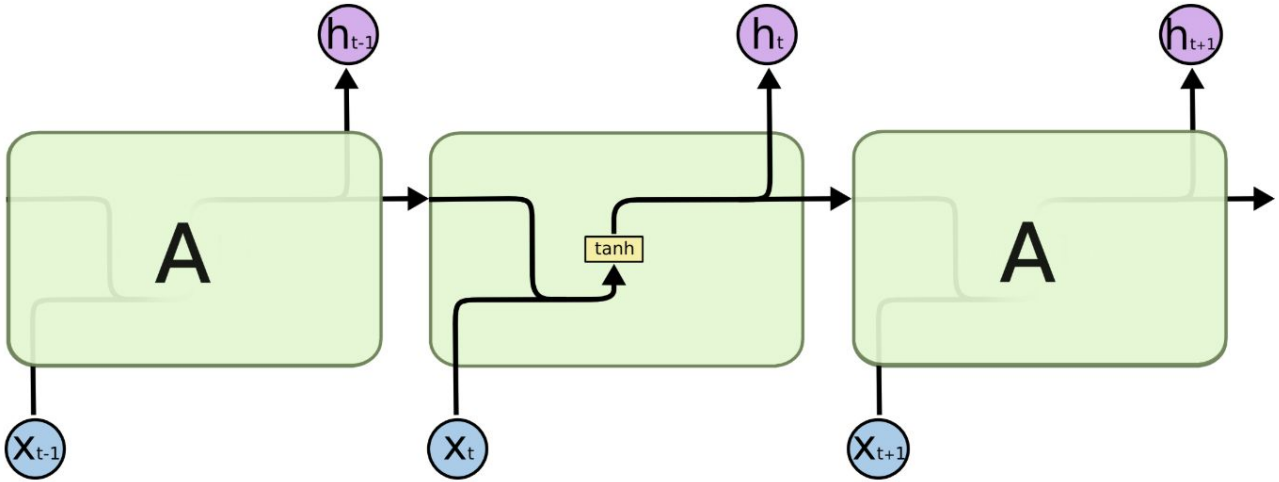
The cell state is updated as:

$$C_t = f_t \cdot C_{t-1} + i_t \cdot \tilde{C}_t$$

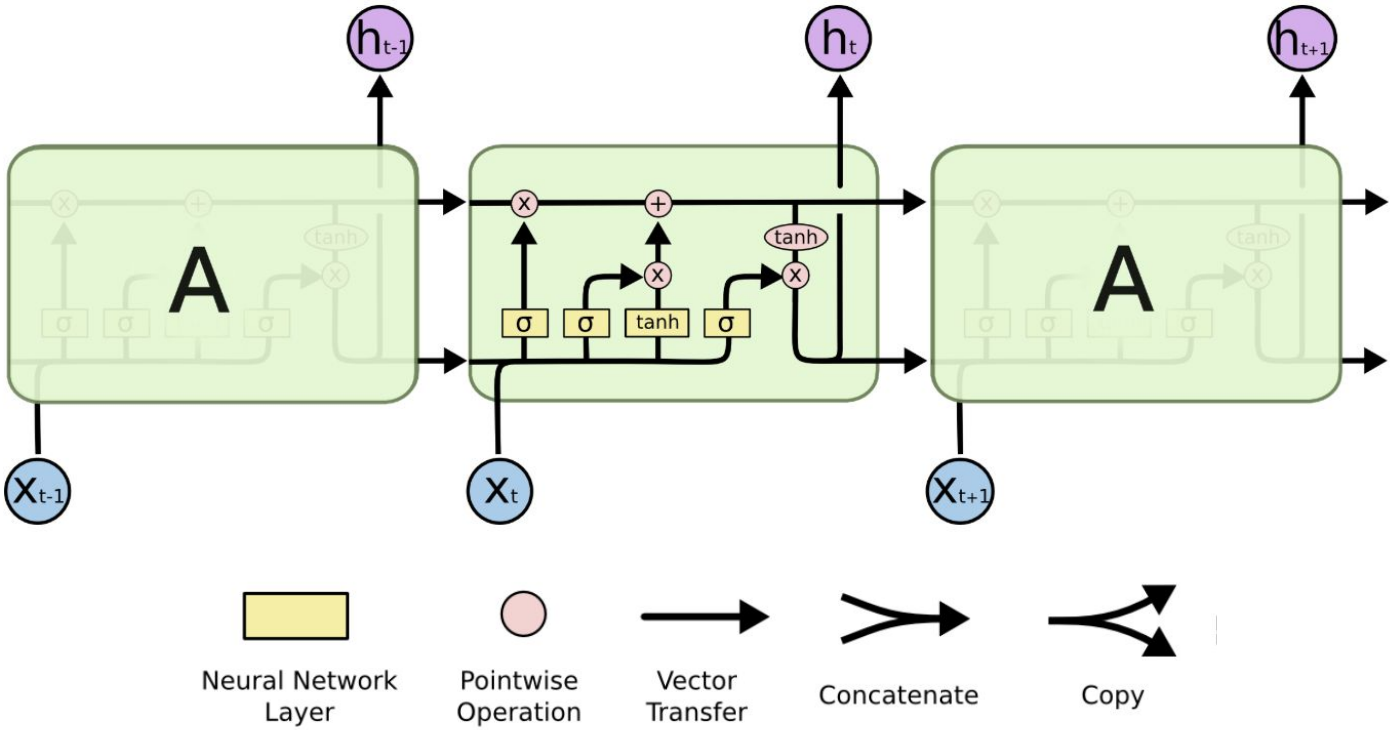


- All gates work on same inputs but learn different **weights** hence different decisions per gate
- Training LSTM = learning **which info to forget, store, or output** for the task

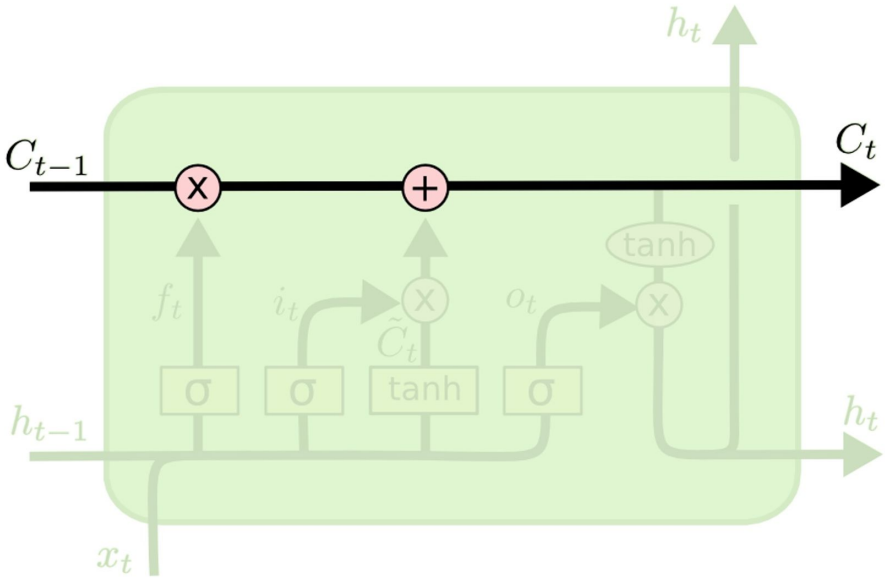
RNN cell contains a single layer



LSTM cell contains 4 interacting layers

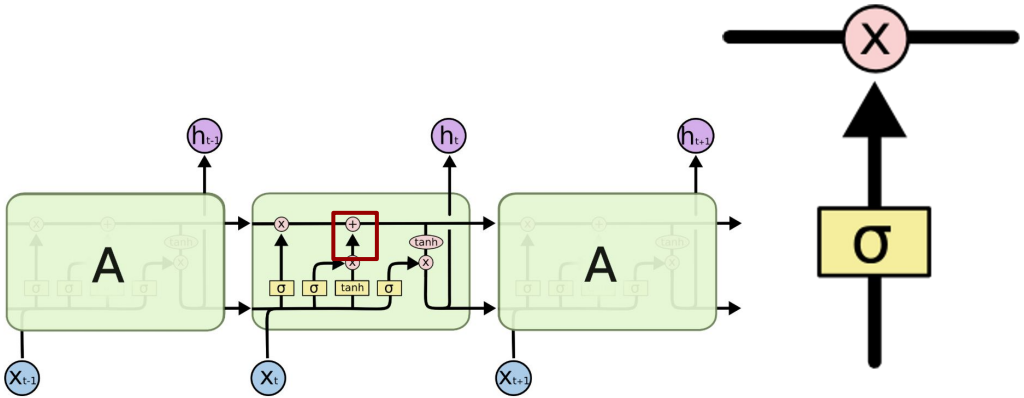


LSTM Cell State



- carries information across timesteps with minimal modification
- allows information (and gradients) to flow more easily compared to standard RNNs.

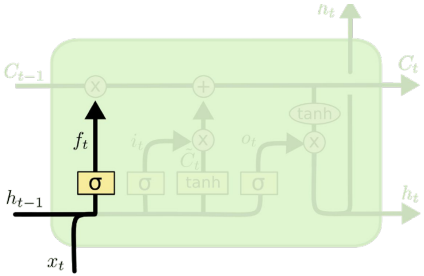
Gates



- Gates are a way to optionally let information through. They are composed out of a sigmoid neural net layer and a pointwise multiplication operation.
- Result of sigmoid can allow the information in the vector to pass through or to block it
    - Sigmoid function forces values in vector to be close to 0 or 1

Forget Gate

$$f_t = \sigma\left(W_f \cdot [h_{t-1}, x_t] \; + \; b_f\right)$$



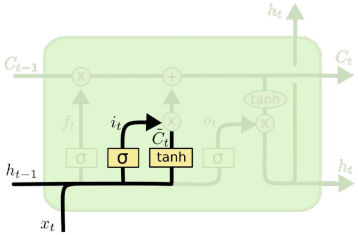
- The forget gate produces a vector of values between 0 and 1
- Each value controls how much of each component of the previous cell state is kept

Interpretation:

- **1 → keep completely**
- **0 → forget completely**
- **values in between → partial retention**

Input Gate

$$i_t = \sigma\left(W_i \cdot [h_{t-1}, x_t] \; + \; b_i\right)$$
$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] \; + \; b_C)$$



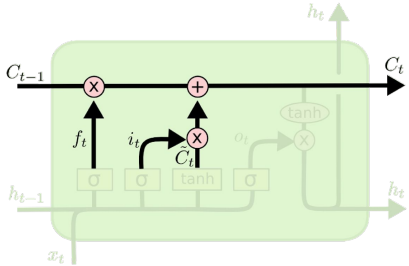
The second step in an LSTM is to decide **what new information to store in the cell state**.

This process has **two components**:

- Input Gate Layer
- Candidate Cell State

Updating C

$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$



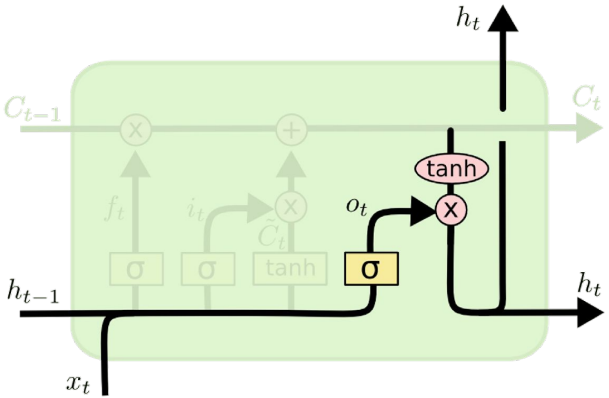
Update the previous cell state C_{t-1} to produce the new cell state C_t.

This step combines the decisions made by:

- the **forget gate**
- the **input gate**
- the **candidate cell state**

Output Gate

- Decide what information from the **cell state** should be exposed as the **output (hidden state)**.
- The hidden state carries this filtered information forward.

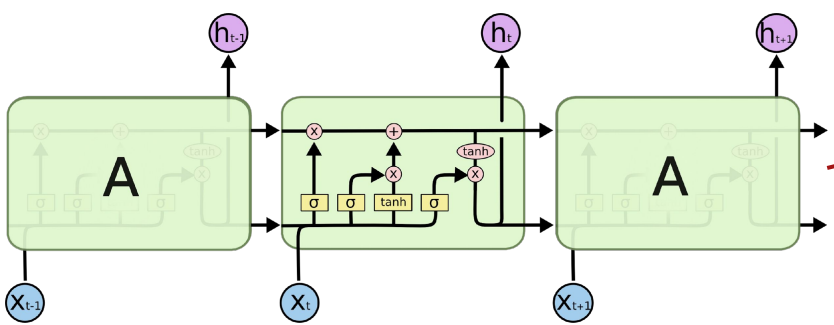


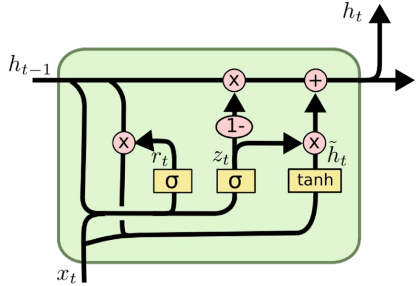
$$o_t = \sigma\left(W_o\left[h_{t-1}, x_t\right] + b_o\right)$$

$$h_t = o_t * \tanh\left(C_t\right)$$

Beyond the LSTM Architecture

LSTM Variants – Gated Recurrent Unit





$$z_t = \sigma\left(W_z \cdot [h_{t-1}, x_t]\right)$$

$$r_t = \sigma\left(W_r \cdot [h_{t-1}, x_t]\right)$$

$$\tilde{h}_t = \tanh\left(W \cdot [r_t * h_{t-1}, x_t]\right)$$

$$h_t = (1 - z_t) * h_{t-1} + z_t * \tilde{h}_t$$

- A GRU is a simplified version of an LSTM
- merges forget and input gates into a single update gate
- removes the separate cell state
- making it lighter while still capturing long-term dependencies.

LSTM Variants – Gated Recurrent Unit

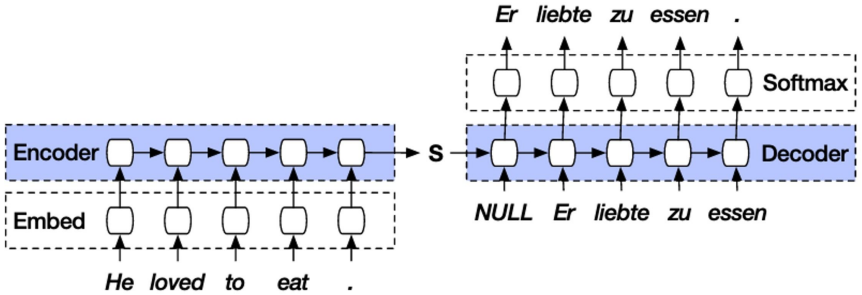
- **GRU:**
  - Uses **hidden state (h_t)** only.
  - **Update gate** controls how much past info to keep vs update with new info.
  - Simplified, but still preserves long-term dependencies in h_t.

Feature	GRU	LSTM
Gates	2 (update + reset)	3 (forget, input, output)
States	Hidden state only	Hidden + cell state
Parameters	Fewer → faster	More parameters → more computation
Performance	Often similar to LSTM	Slightly more flexible for long sequences

Example LSTM Applications: Neural Machine Translation

LSTMs are used in Neural Machine Translation to encode a source sentence into a context representation

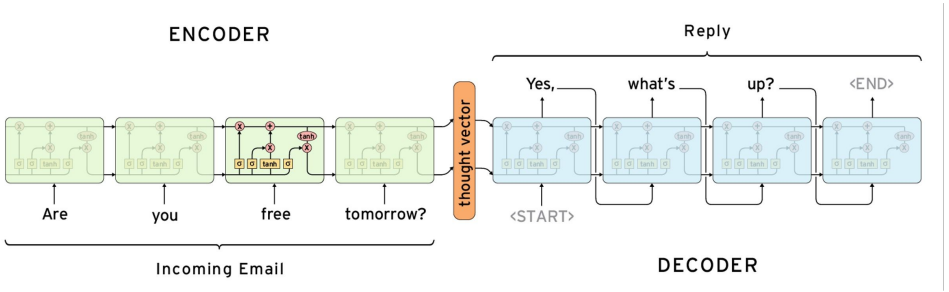
- decode it step-by-step into a target language
- preserving long-range dependencies like subject, tense, and structure.
- LSTM memory helps retain important information like subject, tense, and word order



https://smerity.com/articles/2016/google_nmt_arch.html

Example LSTM Applications: Chatbots

- LSTMs are used in chatbots to understand user input as a sequence of words
- maintain conversational context (memory)
- Context tracking: Remembers previous messages in the conversation
- Memory of long dependencies: Helps maintain topic, intent, and entities across dialogue



Comparison Between Architectures

Model	Use when	Strength	Weakness
Vanilla Recurrent Neural Network	Short sequences, simple temporal patterns	Simple, lightweight	Poor at long-term dependencies
Gated Recurrent Unit	Medium to long sequences	Simpler than LSTM, fewer parameters	Slightly less expressive than LSTM
Long Short-Term Memory Network	Long sequences, long-range dependencies	Strong memory via cell state	More complex, heavier
Convolutional Neural Network	Spatial data (images) or local patterns in sequences	Fast, parallelizable, good at local feature extraction	Not inherently sequential (limited temporal modeling)

Next..

- **RNN** → “walk sequentially, remember via hidden state”
- **LSTM/GRU** → “walk sequentially, keep a notebook (cell/hidden state) for long-term memory”
- **Transformer** → “look at all tokens at once, pay attention to what matters most”

Model	Memory	Parallelism	Long-term dependency	Pros	Cons
RNN	Hidden state	Sequential	Poor	Simple	Vanishing gradient, slow
LSTM / GRU	Hidden + cell state	Sequential	Good	Handles long-term memory	Still sequential, slower
Transformer	Attention (no recurrence)	Fully parallel	Excellent	Handles long-range deps, scalable	Memory-heavy, data-hungry

Lab Work

- Check the notebook and go through it to see practical examples and review the theory as well.

APPENDIX